StoryMe

AI Identity Scene Consistency Pipeline

Version	v2 — Complete Redesign
Document type	Design Decision Record + Cost Analysis
Date	14 May 2026
Author	Engineering / StoryMe Platform
Status	Approved for Implementation
File	storyme/tests/playground/scripts/ai/notes/ai_identity_scene_consistency_v2.pdf

This document captures the complete design rationale, pipeline architecture, model selection decisions, cost analysis, and Azure deployment prerequisites for version 2 of the AI Identity Scene Consistency pipeline. Version 1 was diagnosed with five distinct failure layers — including a fake random-number similarity metric — and this document records every decision made to correct those failures. It serves as the authoritative reference for anyone implementing, reviewing, or extending this pipeline.

Table of Contents

Table of Contents . . . 2
1. Problem Statement . . . 3
2. Solution Architecture . . . 4
2.1 Stage-by-stage description . . . 4
3. Design Decisions . . . 6
4. Models Used . . . 8
4.1 Key configuration parameters . . . 8
5. Cost Analysis . . . 9
5.1 Azure Blob Storage — ML model files . . . 9
5.2 Replicate API — InstantID on L40S . . . 9
5.3 Replicate calls per 18-page book . . . 9
5.4 Per-page and per-book cost . . . 9
6. Azure Deployment Prerequisites . . . 11
6.1 Azure infrastructure . . . 11
6.2 Python dependencies . . . 11
6.3 New codebase files required . . . 11
6.4 Environment variables . . . 11
7. Assumptions and Limitations . . . 13
7.1 Assumptions . . . 13
7.2 Known limitations . . . 13
8. Implementation Checklist . . . 15
9. Sources and References . . . 16

1. Problem Statement

The original **ai_identity_scene_consistency.py** script was analysed line by line. Five distinct failure layers were identified, each independently capable of making the pipeline produce wrong faces. Together they guaranteed failure.

Failure 1 — Critical	**Fake similarity metric** compute_identity_similarity() returns np.random.uniform(0.72, 0.92). The entire retry loop, threshold gate, and 'Similarity threshold satisfied' log message are theater. No face measurement ever occurred. Page 1 scoring 0.7849 and Page 2 scoring 0.9175 are two dice rolls.
Failure 2 — Critical	**Face crop is not face detection** create_face_crop() crops a fixed rectangle: 20%-80% width, 10%-75% height. No face detection. No landmark alignment. The crop may contain minimal face if the child is off-center. A face recognition model expects a normalized, aligned 112x112 crop — not an arbitrary rectangle.
Failure 3 — Critical	**Wrong generation model architecture** Passing images as 'reference' to a base diffusion model (FLUX/SD) routes them through CLIP's image encoder. CLIP captures semantic style, not facial biometrics. Research shows CLIP produces ~80% cosine similarity between different people's faces — it is nearly blind to individual identity differences. No amount of prompting fixes a fundamental architecture mismatch.
Failure 4 — High	**Placeholder model name** MODEL_NAME = "google/nano-banana" is not a real Replicate model identifier. Either the model being called was injected elsewhere, or every call hit a generic model never designed for identity preservation.
Failure 5 — High	**Previous-page feedback loop compounds drift** Adding previous_page to each scene's reference list anchors generation to an already incorrect face. If Page 1 is wrong, Page 2 is anchored to wrongness. Each successive page drifts further from the original photo, not closer. The 0.9175 score on Page 2 is a random number — but if it were real, it would indicate similarity to Page 1's wrong face, not to the original photo.

Root cause summary: The pipeline attempted to solve a biometric identity preservation problem using a semantic image generation model, and measured success with a random number. These are three mismatched tools for the same problem.

2. Solution Architecture

The redesigned pipeline has five sequential stages. Stages 1 and 2 run once per child photo. Stage 3 loops over every story page. Stage 4 is a fallback that only activates when all three generation attempts fail the ArcFace threshold. Stage 5 commits each page.

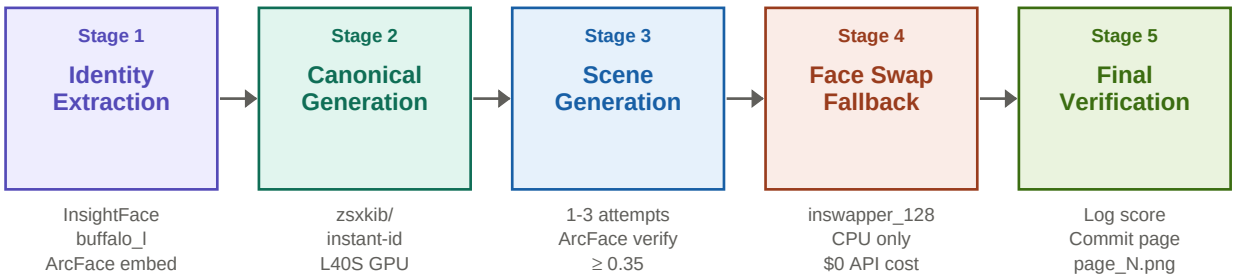


Figure 1 — Simplified five-stage pipeline. Arrows show data flow. Stage 4 (face swap fallback) is a dashed optional path.

2.1 Stage-by-stage description

Stage 1 — Identity extraction

Runs once per child photo. InsightFace buffalo_l detects the face, extracts 5-point facial landmarks, and computes a 512-dimensional ArcFace embedding (the biometric fingerprint). The embedding is serialized to face_ref.pkl. An aligned 112x112 face crop is saved for use as the generation reference image. If no face is detected the pipeline hard-fails with a clear error before any Replicate API call is made.

Stage 2 — Canonical character generation

Runs once per child. Calls zsxkib/instant-id on Replicate with the aligned face crop and a front-facing portrait prompt. The output is verified with ArcFace cosine similarity against face_ref.pkl. Threshold: 0.40 (real to animated, front portrait). Up to 3 attempts. The canonical character is a style-locked portrait used as a visual anchor — it is NOT passed as a reference into page generation (which would cause style drift).

Stage 3 — Per-page scene generation

Runs for each story page. Builds the full prompt (style block + character block + scene block). Calls InstantID with the aligned face crop and scene prompt. Verifies with ArcFace at threshold 0.35 (lower than canonical because scenes involve more pose variation and occlusion). Seed formula: BASE_SEED + (page_index * 100) + (attempt * 10) — every retry genuinely explores a different generation. The previous page output is NOT passed as a reference — this prevents identity drift compounding.

Stage 4 — Face swap fallback

Triggers only when all 3 generation attempts fail the 0.35 threshold. Uses InsightFace inswapper_128 (CPU, no Replicate API cost) to detect the face in the best-scoring generated scene and swap the reference face in. paste_back=True handles background blending. No GFPGAN in v1 — this is reserved for a future enhancement pass.

Stage 5 — Final verification and commit

Runs ArcFace similarity one final time and logs the real score. This creates an auditable quality record per page. The page is committed as page_N.png and previous_page is updated to this path for logging purposes only — it is not fed back into generation.

3. Design Decisions

Every decision in this section was made deliberately. The rationale is recorded here so future engineers understand why the pipeline is structured this way.

DD-01	**Replace CLIP-based similarity with ArcFace cosine similarity**	CLIP encodes semantic style. It produces ~80% cosine similarity between completely different people's faces. ArcFace is trained specifically with additive angular margin loss to maximise inter-identity separation. InsightFace normed_embedding is already L2-normalised so similarity = dot product.	*ArcFace cosine similarity via InsightFace buffalo_l*
DD-02	**Set threshold at 0.35 for page generation, 0.40 for canonical**	A real photograph converted to an animated storybook cartoon style reduces ArcFace similarity by roughly 0.15-0.25 compared to same-domain (real-to-real) matching where same-person threshold is 0.60. 0.35 is calibrated for the real-to-animated cross-domain gap with significant pose variation. 0.40 for canonical because it is a controlled front-facing portrait with less pose variation.	*0.35 (page), 0.40 (canonical)*
DD-03	**Use InsightFace FaceAnalysis for face detection and crop**	The fixed-percentage rectangular crop (20-80% width, 10-75% height) is unreliable. InsightFace buffalo_l uses a RetinaFace detector with 5-point landmark prediction. The aligned crop is normalized to 112x112 in the exact format ArcFace was trained on.	*InsightFace FaceAnalysis, buffalo_l model*
DD-04	**Use InstantID (zsxkib/instant-id) as the generation model**	InstantID uses a learned IdentityNet that conditions the diffusion process on ArcFace face embeddings and facial landmark keypoints — not CLIP. This is the fundamental architectural difference that enables biometric-level identity conditioning. IP_ADAPTER_SCALE=0.85 weights identity heavily. SDXL weights: dreamshaper-xl-turbov2 for animated/storybook style.	*zsxkib/instant-id:latest on Replicate L40S GPU*
DD-05	**Remove previous_page from generation references**	Each page having the previous page as a reference anchors generation to an already-drifted face. The only reference that matters for identity is the aligned face crop from Stage 1. Style consistency across pages is achieved through fixed prompt blocks (style block + character block), not image references.	*Removed. previous_page logged only, never fed into generation.*

DD-06	**Vary seed per page and per attempt**	A fixed seed of 12345 for all generations means retries do not explore the generation space — they reproduce identical outputs. Formula: BASE_SEED + (page_index * 100) + (attempt * 10) ensures every attempt is genuinely different while remaining reproducible across full pipeline runs.	*seed = BASE_SEED + (page * 100) + (attempt * 10)*
DD-07	**Use InsightFace inswapper_128 as fallback, not seamlessClone**	The playground tests inswapper independently of the main codebase's face_pipeline_service.py (seamlessClone). inswapper_128 uses a trained face-swap model that handles geometry warping, lighting adaptation, and edge blending in one pass. seamlessClone requires manual mask preparation and is sensitive to pose angle. inswapper is more autonomous.	*InsightFace inswapper_128.onnx, CPU inference*
DD-08	**Store ML models in Azure Blob Storage ml-models container**	InsightFace auto-download from GitHub Releases is unreliable for inswapper_128 (known failures documented in GitHub issues #2294, #2335, #2385). Azure Blob Storage, already in the StoryMe architecture, is the correct model registry. One-time upload via scripts/upload_ml_models.py. Per-startup download from Blob at ~15s for 1GB on Azure internal network.	*Azure Blob container: ml-models, file: inswapper_128.onnx (554 MB)*
DD-09	**Verify SHA256 on every model download**	Canonical SHA256 of inswapper_128.onnx: e4a3f08c753cb72d04e10aa0f7dbe3deebbf39567d4ead6dce08e98aa49e16af (554 MB). A partial or corrupt download must be detected and re-downloaded before any pipeline run. A corrupt model produces silent failures that are extremely difficult to debug.	*SHA256 verified after every download, before model load*
DD-10	**Use /tmp/storyme_models/ as local model cache on Azure App Service**	/home/ on App Service persists across restarts but is wiped on redeployment. /tmp/ is wiped on restart but is local to the container, has fast I/O, and avoids race conditions with multiple instances. At ~15s re-download per cold start from same-region Blob, the cost is acceptable for a background job service. Model path is provided via ml_model_manager.py, not hardcoded.	*/tmp/storyme_models/ (App Service) | PLAYGROUND_DIR/models/ (local dev)*

4. Models Used

All models are referenced by their exact identifiers. This section is the authoritative model registry for this pipeline.

Model	Identifier	Purpose	Source	Size	Inference
InstantID	zsxkib/instant-id:latest	Identity-preserving scene and portrait generation	Replicate API	N/A (cloud)	L40S GPU $0.000975/sec
buffalo_l	insightface buffalo_l	Face detection + ArcFace embedding extraction	InsightFace auto-download on first run	~420 MB	CPU (onnxruntime)
inswapper_128	inswapper_128.onnx	Face swap fallback (Stage 4 only)	Azure Blob Storage ml-models container	554 MB	CPU (onnxruntime)
ArcFace (built into buffalo_l)	faces[0].normed_embedding	Identity verification (512-dim cosine similarity)	Part of buffalo_l pack	Included	CPU

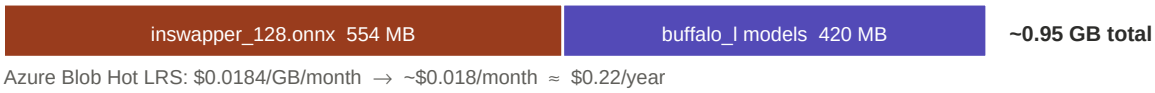
4.1 Key configuration parameters

Parameter	Value	Rationale
`IP_ADAPTER_SCALE`	`0.85`	High identity weighting. 1.0 = full identity lock (less creative freedom). 0.85 balanc
`CONTROLNET_CONDITIONING_SCALE`	`0.80`	ControlNet pose conditioning weight. Controls how strictly the face keypoints are
`IMAGE_WIDTH / IMAGE_HEIGHT`	`768 x 768`	Good quality for storybook pages. 1024x1024 increases cost ~40% with margina
`NUM_INFERENCE_STEPS`	`30`	Standard quality/speed balance for SDXL. 20 is faster but lower quality. 50 is dim
`GUIDANCE_SCALE`	`5.0`	CFG scale. Lower = more model freedom. 5.0 is calibrated for animated storyboo
`SDXL_WEIGHTS`	`dreamshaper-xl-turbov2`	Best animated/storybook style among available SDXL checkpoints on Replicate.
`SIMILARITY_THRESHOLD (page)`	`0.35`	Calibrated for real-to-animated cross-domain ArcFace similarity gap (~0.15-0.25
`SIMILARITY_THRESHOLD (canonical)`	`0.40`	Slightly higher: canonical is a controlled front-facing portrait with less pose variati
`MAX_RETRIES`	`3`	3 attempts per page. Beyond 3 the probability of success does not improve mear
`BASE_SEED`	`42`	Deterministic but varied per page: seed = 42 + (page * 100) + (attempt * 10).

5. Cost Analysis

5.1 Azure Blob Storage — ML model files

Model files are stored once in the Azure Blob Storage ml-models container. Egress within the same Azure region is free, making per-startup downloads essentially zero-cost beyond the initial storage fee.



Azure Blob Hot LRS: $0.0184/GB/month → ~$0.018/month ≈ $0.22/year

Item	Detail	Monthly cost	Annual cost
inswapper_128.onnx storage	554 MB × $0.0184/GB	$0.010	$0.122
buffalo_l models storage	420 MB × $0.0184/GB	$0.008	$0.096
Total model storage	~974 MB ≈ 1 GB	$0.018	$0.218
Download operations	< 10 ops per startup	~$0.000	~$0.000
Egress (same Azure region)	Free — internal network	$0.000	$0.000
TOTAL		**~$0.02/month**	**~$0.22/year**

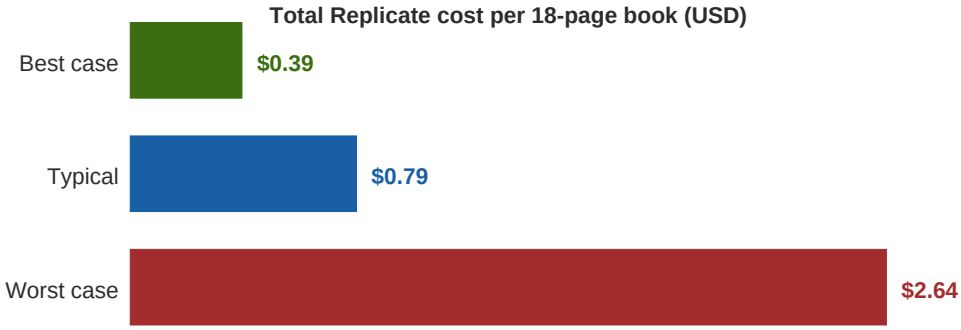
5.2 Replicate API — InstantID on L40S

Replicate bills per second of GPU compute. InstantID runs on Nvidia L40S GPU at **$0.000975/second**. Typical completion time is **~26 seconds** (confirmed from Replicate model page). Cold start (model not warm) adds 30-60 seconds billed once per book generation session.

5.3 Replicate calls per 18-page book

Call type	Count (best)	Count (typical)	Count (worst)	Notes
Cold start (warm-up)	1	1	1	Billed once per session. +30-60s.
Canonical character	1	1-2	3	Fails if ArcFace < 0.40
Page generation (×18 pages)	18	~27	54	1-3 attempts per page
Face swap fallback	0	0	$0	CPU only — no Replicate cost
TOTAL API CALLS	**19**	**~28**	**57**	

5.4 Per-page and per-book cost



Scenario	Calls	Approx time	Replicate cost	Blob storage
Best — all pages pass on 1st attempt	19	~7 min	**$0.39**	~$0.00
Typical — avg 1.5 attempts per page	~28	~13 min	**$0.79**	~$0.00
Worst — all 3 retries, 45s each	57	~43 min	**$2.64**	~$0.00

Business perspective: At a book price of $15-$25, Replicate generation cost represents 1.6%-17.6% of revenue (best to worst case). The typical scenario at ~$0.79 per book represents 3%-5% of a $15-$25 price point — well within a viable margin. Azure Blob model storage adds $0.22/year regardless of volume — effectively zero.

6. Azure Deployment Prerequisites

Everything in this section must be in place before the pipeline can run on Azure App Service. These are hard requirements, not suggestions.

6.1 Azure infrastructure

Requirement	Detail	Status in StoryMe
Azure App Service (Linux)	Python 3.11 runtime. B2 or higher SKU for CPU inference headroom. Insightface runs entirely on CPU.	Existing — verify SKU
Azure Blob Storage account	General Purpose v2, Hot tier, LRS redundancy. Same region as App Service.	Existing
Blob container: ml-models	New container alongside existing ones. Private access. Stores inswapper_128.onnx (~554 MB).	NEW — create before deploy
Blob container: existing assets	storyme-assets or equivalent — no change.	Existing
Same Azure region	App Service and Blob Storage MUST be in the same region to avoid egress charges on model download.	Verify
Replicate API token	REPLICATE_API_TOKEN environment variable in App Service configuration.	Existing

6.2 Python dependencies

Add to requirements.txt (backend). These are net-new dependencies vs the current production stack:

Package	Version (minimum)	Purpose
`insightface`	`>=0.7.3`	Face detection (buffalo_l), ArcFace embedding, inswapper_128
`onnxruntime`	`>=1.16.0`	CPU inference backend for InsightFace models (NO GPU required)
`opencv-python-headless`	`>=4.8.0`	Image I/O and processing (headless = no display dependency)
`numpy`	`>=1.24.0`	ArcFace cosine similarity computation (likely already installed)
`requests`	`>=2.31.0`	Downloading inswapper_128.onnx from HuggingFace for local dev
`replicate`	`>=0.25.0`	Replicate API client (already in stack — verify version)
`azure-storage-blob`	`>=12.19.0`	Blob Storage download for ml-models (already in stack)

6.3 New codebase files required

File path	Type	Purpose
`backend/ml_model_manager.py`	NEW	ensure_models(model_dir, logger): downloads models from Blob on startup. Verifies
`scripts/upload_ml_models.py`	NEW	One-time developer script. Downloads inswapper_128.onnx from HuggingFace, verif
`tests/playground/scripts/ai/ai_identity_scene_consistency_v2.py`	REWRITE	Complete replacement of v1. Implements all 5 stages with real ArcFace similarity.
`backend/server.py`	MODIFY	Add ml_model_manager.ensure_models() call in FastAPI lifespan startup event.
`backend/face_pipeline_service.py`	MODIFY (minor)	Update inswapper_128 path to use ml_model_manager.get_model_path() instead of

6.4 Environment variables

Variable	Required for	Set in
`REPLICATE_API_TOKEN`	All Replicate API calls	App Service config (existing)
`AZURE_STORAGE_CONNECTION_STRING`	Blob download of inswapper_128.onnx	App Service config (existing)
`ML_MODELS_BLOB_CONTAINER`	ml_model_manager.py container name	App Service config (NEW) — default: ml-models
`ML_MODELS_LOCAL_DIR`	Local model cache path override	Optional — defaults to /tmp/storyme_models/
`INSWAPPER_SHA256`	Hash verification on download	Optional — hardcoded fallback in ml_model_manager.py

7. Assumptions and Limitations

7.1 Assumptions

A01	**Input photo quality** The child's face must be clearly visible, unobstructed, well-lit, and at least ~100px wide in the input image. InsightFace buffalo_l will hard-fail to detect faces smaller than this. The pipeline raises FaceNotDetectedError and does not proceed.
A02	**Single face in photo** The pipeline selects the largest detected face. If multiple children are in the photo, the largest face is used. This is the expected use case (one reference child per book).
A03	**ArcFace cross-domain threshold calibration** The 0.35 threshold is calibrated based on the expected reduction in ArcFace cosine similarity when converting a real photo to an animated cartoon style (typically 0.15-0.25 reduction vs real-to-real same-person threshold of 0.60). This threshold may need empirical adjustment based on actual test results.
A04	**InsightFace non-commercial license** InsightFace buffalo_l and inswapper_128 are available for non-commercial research use only under InsightFace's license terms. Commercial deployment requires a commercial license from InsightFace. Verify before production launch.
A05	**Replicate uptime and cold start** Cold start time (30-60s) is an estimate based on typical L40S model load times. Replicate SLAs do not guarantee cold start bounds. In production, high-frequency usage will keep the model warm and eliminate cold starts.
A06	**Azure Blob egress** Model download from Blob is costed at $0 assuming App Service and Blob Storage are in the same Azure region. Cross-region egress is $0.087/GB — a 1GB download would cost $0.087 per cold start.
A07	**18-page book** All cost calculations are based on an 18-page story. The pipeline supports any number of pages via story.json. Canonical generation is always 1 pass regardless of page count.

7.2 Known limitations

L01	**Side-profile faces reduce ArcFace accuracy** ArcFace similarity drops significantly for faces turned >45 degrees from frontal. Scene prompts should be written to keep the character's face visible. GFPGAN post-enhancement is planned for v2 to improve fallback quality.
L02	**No GFPGAN in v1** The face-swap fallback does not include a face restoration step in v1. The inswapper output may have visible blending artifacts. This is logged as a warning. GFPGAN integration is the highest-priority v2 enhancement.

L03	**Worst-case latency** 57 Replicate calls at 45s each = ~43 minutes. This is only reached if every page fails all 3 attempts. In practice, well-lit frontal child photos should produce typical-case results (~13 minutes). Latency is acceptable for a background async job with flipbook preview.
L04	**InsightFace commercial license** Production deployment with buffalo_l and inswapper_128 requires confirming commercial license terms with InsightFace. The open-source models are for research only per their license.

8. Implementation Checklist

Use this checklist when implementing and deploying the pipeline. Complete items in order — each phase depends on the previous.

Phase 1 — Playground (local Windows)

- [ ] pip install insightface onnxruntime opencv-python replicate requests
- [ ] Run script with --name nikshay: buffalo_l auto-downloads to PLAYGROUND_DIR/models/
- [ ] Script calls ensure_inswapper_model(): downloads 554MB from HuggingFace, verifies SHA256
- [ ] Confirm face detected from nikshay_image.jpeg (logs show bbox + embedding shape)
- [ ] Confirm canonical character generated (ArcFace score logged — not random)
- [ ] Run 2-page test: confirm per-page ArcFace scores are real and sensible (0.3-0.6 range)
- [ ] If all pages hit fallback: check IP_ADAPTER_SCALE, verify aligned crop quality
- [ ] Run full 18-page story: confirm all pages commit and scores are logged

Phase 2 — One-time model upload

- [ ] Create Azure Blob container: ml-models (private access, same region as App Service)
- [ ] Run scripts/upload_ml_models.py: downloads inswapper_128.onnx, verifies SHA256, uploads
- [ ] Confirm file visible in Azure Portal: ml-models/inswapper_128.onnx (554 MB)
- [ ] Add ML_MODELS_BLOB_CONTAINER=ml-models to App Service environment variables

Phase 3 — Production code (backend)

- [ ] Create backend/ml_model_manager.py with ensure_models() and get_model_path()
- [ ] Add ensure_models() call in FastAPI lifespan startup event in server.py
- [ ] Update face_pipeline_service.py to get inswapper path from ml_model_manager
- [ ] Add insightface and onnxruntime to requirements.txt
- [ ] Deploy to App Service: confirm startup logs show model download + SHA256 OK
- [ ] Run end-to-end book generation on App Service: confirm ArcFace scores in logs

Phase 4 — Quality validation

- [ ] Generate 5 test books with different child photos
- [ ] Review per-page ArcFace scores in logs: typical range should be 0.35-0.55
- [ ] Count fallback activations: if >30% of pages hit fallback, lower threshold to 0.30

- Visual review: confirm character face is consistent and recognisable across pages
- Review flipbook preview UX: confirm user can see all pages before download decision

Phase 5 — Future enhancements (not in v1)

- Integrate GFPGAN face restoration in Stage 4 fallback for better blend quality
- Add InsightFace commercial license for production legal compliance
- Implement prompt emotion conditioning: derive emotion token from page scene text
- Add per-book cost tracking: log total Replicate spend per book to Azure Table Storage
- Evaluate FLUX Kontext as alternative generation model when stable on Replicate

9. Sources and References

Ref	Title	URL / Source
S01	InstantID paper (arXiv:2401.07519)	https://arxiv.org/abs/2401.07519
S02	zsxkib/instant-id on Replicate	https://replicate.com/zsxkib/instant-id
S03	Replicate hardware pricing	https://replicate.com/ (L40S: $0.000975/sec)
S04	Replicate L40S GPU announcement	https://replicate.com/blog/nvidia-l40s-gpus-are-here
S05	InsightFace inswapper README	https://github.com/deepinsight/insightface/blob/master/examples/in_swapper/README.md
S06	inswapper_128.onnx SHA256 verification	HuggingFace: Aitrepreneur/insightface — SHA256: e4a3f08c...
S07	Azure Blob Storage pricing	https://azure.microsoft.com/pricing/details/storage/blobs/ (Hot LRS: $0.0184/GB/month)
S08	CLIP face recognition limitation	arXiv:2411.12319 — CLIP produces ~80% cosine sim between different faces
S09	ArcFace loss paper	Deng et al. 2019, IEEE CVPR — Additive Angular Margin Loss
S10	InsightFace inswapper download issues	GitHub issues #2294, #2335, #2385 — auto-download unreliable from GitHub Releases